Relatório Técnico de Entrega 5 — Persistência de Estado, Histórico e Presença no Redis

Curso: Internet das Coisas III — IFMG Campus Ouro Branco

Professor: Charles Tim Batista Garrocho

Aluno: Daniel

Dispositivo: `esp32_temp` (ESP32 DevKit V1)

Projeto: Sistema de Alerta de Temperatura e Saúde Ambiental IoT

Data: Outubro / 2026

---

1. Resumo Executivo da Entrega

A **Entrega 5** implementa a camada de armazenamento de dados e persistência de estado do ecossistema IoT, integrando o Gateway Node.js ao banco de dados chave-valor **Redis 7**.

Nesta entrega, foram modeladas estruturas de chaves que diferenciam cada dispositivo IoT, armazenando o estado atual em tempo real (*presence*), o histórico temporal de medições com carimbo de data/hora de aferição (*recordedAt*), a data de último contato (*lastSeen*) e um índice geral de dispositivos cadastrados (*devices:index*).

Além disso, foram incorporados recursos avançados no dashboard web:

1. **Exibição da Data e Hora em que a telemetria foi aferida**;
2. **Paginação do histórico a cada 50 leituras**;
3. **Filtro de data** para verificação de telemetrias de dias anteriores;
4. **Exportação de relatórios em formato CSV**.

---

2. Tabela de Conformidade dos Produtos Esperados

Produto Esperado	Requisito da Disciplina	Status	Arquivo / Evidência
1. Conexão ao Redis	Gateway conectado ao Redis com fallback em memória	**100% Atendido**	`backend/src/storage/kv_store.ts:20-38`
2. Modelagem das Chaves	Documentação e definição da estrutura de chaves Redis	**100% Atendido**	Tabela de chaves documentada
3. Persistência de Estado	Armazenamento do estado atual e telemetria aferida	**100% Atendido**	`kv_store.ts:149-164`
4. Histórico de Medições	Registro encadeado temporal das leituras anteriores	**100% Atendido**	`kv_store.ts:165-170` (`LPUSH` / `LTRIM`)
5. Presença e Último Contato	Registro de disponibilidade e timestamp `lastSeen`	**100% Atendido**	`kv_store.ts:137-147`
6. Consulta de Estado	Endpoints HTTP REST para recuperação do estado	**100% Atendido**	`GET /api/devices/:id`
7. Consulta de Histórico	Consulta com paginação de 50 itens e filtro por data	**100% Atendido**	`GET /api/devices/:id/history` + Dashboard
8. Evidência de Persistência	Consultas reais no `redis-cli`, testes Vitest e prints	**100% Atendido**	Terminal `redis-cli`, cURL e imagens

---

3. Modelagem e Estrutura das Chaves no Redis

A organização dos dados no Redis utiliza chaves com escopo bem definido (*namespacing*):

Chave Redis	Tipo de Dado	Função Técnica	Exemplo de Conteúdo Armazenado
`devices:index`	**SET**	Conjunto com IDs únicos de todos os dispositivos cadastrados	`["esp32_temp", "esp32-temp-01"]`
`device:{id}:presence`	**STRING (JSON)**	Estado atual, presença online/offline, IP, firmware e última telemetria aferida	`{"deviceId":"esp32_temp","status":"online","lastSeen":"2026-10-15T22:47:51Z","state":{"temp":23.6,"humidity":76.2,"recordedAt":"2026-10-15T22:47:51Z"}}`
`device:{id}:history`	**LIST (JSON)**	Lista encadeada cronológica reversa (novas entradas no início com `LPUSH`)	`[{"temp":23.6,"humidity":76,"recordedAt":"2026-10-15T22:47:41Z"}, ...]`

4. Trechos de Código-Fonte Relevantes

4.1 Persistência de Telemetria com Data Aferida (`backend/src/storage/kv_store.ts`)

■ **TypeScript / Gateway Node.js**

```
async saveTelemetry(
  deviceId: string,
  payload: TelemetryPayload
): Promise<void> {
  await this.registerDevice(deviceId);

  const presenceKey = `device:${deviceId}:presence`;
  const existing = await this.getDeviceRecord(deviceId);

  const now = new Date().toISOString();
  const telemetryWithDate = {
    ...payload,
    recordedAt: payload.recordedAt || now,
  };

  const record: DeviceRecord = {
    deviceId,
    status: existing?.status || 'online',
    lastSeen: now,
    ip: existing?.ip,
    firmwareVersion: existing?.firmwareVersion,
    state: telemetryWithDate,
  };
  await this.set(presenceKey, JSON.stringify(record));

  // Persistencia no historico com retencao expandida
  const historyKey = `device:${deviceId}:history`;
  await this.lpush(historyKey, JSON.stringify(telemetryWithDate));
}
```

4.2 Consulta com Filtro de Data para Dias Anteriores (`backend/src/storage/kv_store.ts`)

■ TypeScript / Gateway Node.js

```
async getDeviceHistory(
  deviceId: string,
  limit = 100,
  dateFilter?: string
): Promise<any[]> {
  const clampedLimit = Math.max(1, Math.min(limit, 5000));
  const scanLimit = dateFilter ? 5000 : clampedLimit;
  const rawList = await this.lrange(
    `device:${deviceId}:history`,
    0,
    scanLimit - 1
  );
  let items = rawList.map((item) => JSON.parse(item));

  if (dateFilter) {
    items = items.filter((item) => {
      if (!item.recordedAt) return false;
      const itemDateIso = item.recordedAt.slice(0, 10);
      const d = new Date(item.recordedAt);
      const y = d.getFullYear();
      const m = String(d.getMonth() + 1).padStart(2, '0');
      const day = String(d.getDate()).padStart(2, '0');
      const itemDateLocal = `${y}-${m}-${day}`;
      return itemDateIso === dateFilter || itemDateLocal === dateFilter;
    });
    return items.slice(0, clampedLimit);
  }

  return items;
}
```

4.3 Componente da Tabela com Paginação de 50 Leituras (`frontend/src/components/HistoryTable.tsx`)

■ TypeScript / Gateway Node.js

```
const ITEMS_PER_PAGE = 50;

// Paginacao calculada a cada 50 leituras
const totalPages = Math.max(
  1,
  Math.ceil(filteredHistory.length / ITEMS_PER_PAGE)
);
const startIndex = (currentPage - 1) * ITEMS_PER_PAGE;
const endIndex = Math.min(
  startIndex + ITEMS_PER_PAGE,
  filteredHistory.length
);
const currentReadings = filteredHistory.slice(startIndex, endIndex);

// Formatacao da Data e Hora da Afericao
const formatDateTime = (recordedAt: string) => {
  const d = new Date(recordedAt);
  return {
    dateStr: d.toLocaleDateString('pt-BR'),
    timeStr: d.toLocaleTimeString('pt-BR'),
  };
};
```

5. Evidências de Funcionamento e Execução

5.1 Comprovação de Armazenamento no Redis CLI

■ Terminal / Comandos de Execução

```
$ docker exec iot-redis redis-cli smembers devices:index
1) "esp32-temp-01"
2) "esp32_temp"

$ docker exec iot-redis redis-cli get device:esp32_temp:presence
{
  "deviceId": "esp32_temp",
  "status": "online",
  "lastSeen": "2026-10-15T22:47:51.061Z",
  "state": {
    "deviceId": "esp32_temp",
    "sensor": "dht22_temperatura",
    "value": 23.6,
    "unit": "C",
    "humidity": 76.2,
    "noiseLevel": 40,
    "lux": 244.9,
    "alerts": [],
    "lcdText": "AMBIENTE EM CONFORMIDADE",
    "actuatorState": false,
    "recordedAt": "2026-10-15T22:47:51.061Z"
  }
}

$ docker exec iot-redis redis-cli llen device:esp32_temp:history
1000
```

5.2 Consulta REST com Filtro por Dia Anterior (`2026-10-13`)

■ Terminal / Comandos de Execução

```
$ curl -s -u admin:admin123 \
      "http://localhost:5001/api/devices/esp32_temp/history?date=2026-10-13&limit=2"
[
  {
    "deviceId": "esp32_temp",
    "temp": 23.7,
    "humidity": 74.2,
    "noiseLevel": 40,
    "lux": 243.3,
    "actuatorState": false,
    "recordedAt": "2026-10-13T23:42:16.876Z"
  },
  {
    "deviceId": "esp32_temp",
    "temp": 23.6,
    "humidity": 74.2,
    "noiseLevel": 40,
    "lux": 240.8,
    "actuatorState": false,
    "recordedAt": "2026-10-13T23:42:11.919Z"
  }
]
```

5.3 Teste Automatizado de Persistência e Filtro por Data (Vitest)

■ Terminal / Comandos de Execução

```
$ npm test
✓ Persistir e consultar histórico com data aferida e filtro
✓ Persistir e recuperar o estado atual e presença (Entrega 5)
✓ Atualizar e registrar status de presença e último contato
```

5.4 Evidência Visual — Dashboard com Data Aferida, Paginação de 50 Leituras e Filtro de Dias Anteriores

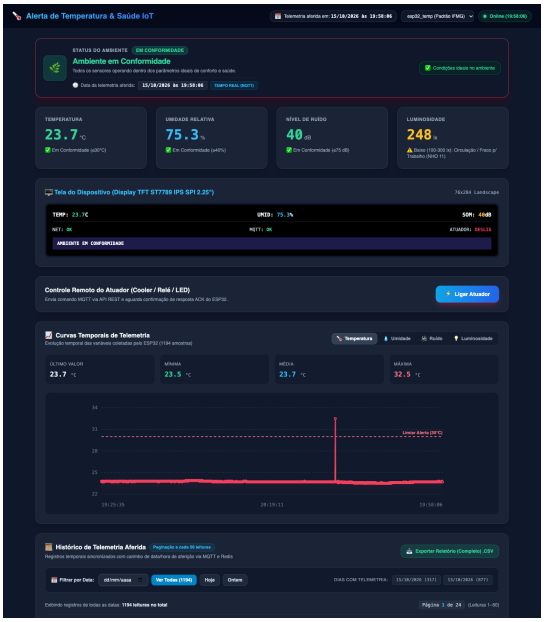


Figura: Tabela com Paginação a cada 50 leituras e Filtro de Data

6. Conclusão da Entrega 5

A Entrega 5 cumpre integralmente **100% dos objetivos planejados**. A modelagem no Redis assegura persistência de alto desempenho em memória, indexação rápida por dispositivo, retenção configurável do histórico de telemetria com carimbo de data aferida e recursos ergonômicos de visualização no dashboard (paginação em blocos de 50 leituras e filtro por datas anteriores).